

Brief Introduction:

Docker is an open platform for developing, shipping, and running applications. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly.

A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another.

Problems:

1. "Not sure why it's not working on your computer, it's working on mine."
2. "Errors: abc module is not compatible; abc module not found..."
3. "It's a pain to install everything from scratch for Linux, Windows, and MacOS, and trying to build the same environment for each OS."
4. "Can't install the package that you used, can you help me out?"
5. "I need more compute power. I could use AWS but it'll take so long just to install all those packages and configure settings just like I have it on my machine."

Solutions:

1. **Virtual Machine (VM)** - VM can run multiple applications on a single server, but
 - a. Each VM requires a separate operating system (OS). Has to purchase a license for every OS.
 - b. The OS utilizes RAM, CPU and storage which is not easy to leverage for powering other applications
 - c. VM is heavy in size and takes a lot of time booting and not great for portability because of the difficulty faced while transferring VM workloads between hypervisors and cloud platforms.
2. **Container** - Container is similar to VM, but the best part of using containers is that
 - a. Containers do not require a separate OS, which means only one OS license is needed.
 - b. Containers are quick to booting and portable-friendly

Benefits:

1. Do not have to worry about managing the requirements of implementing a project on every system.
2. Faster software/ application shipping.
3. Anyone who wants to use the machine learning application, they do not need to install anything except Docker. Avoid setup hassle.
4. The portability and performance benefits offered by containers can make the development process more agile and responsive. It enhances the continuous integration and continuous delivery processes.
5. Can run on any infrastructure and in any cloud.

Security Concerns:

1. Docker containers do not save the data they produce. As soon as the process is finished, the container stops and everything inside of it is removed.
2. [Docker security](#)
3. <https://www.linkedin.com/pulse/key-security-layers-docker-containers-mohit-vaish/?articleId=6618475728398835713>
4. [Docker Security - OWASP Cheat Sheet Series](#)

Workflow

Data Scientist

1. Download Docker Desktop
2. Build Docker Image for the machine learning python program with `docker build -t [app name]:latest -f Dockerfile .`
3. Build container with Docker run command `docker run --name [give a name to container] -d -p 8501:8501 [image id]` *Note: run `docker images` to check the image id.
4. Export the Image as a tar file with `docker save -o early_claim_lapsed_prediction_app_image.tar.gz early_claim_lapsed_prediction_app:latest` ([How to Share Docker Images With Others](#))

5. Pass Docker Image to IT Team to host in the server (Can be Windows and Linux)

IT Team

6. Download Docker Desktop
7. Open a terminal/ Command Prompt
8. Load the image with "docker load < early_claim_lapsed_prediction_app_image.tar.gz"
9. Build a new container with docker run "docker run --name [container_name] --mount source=[volume_name], target=/app -d --restart unless-stopped -p [host_port]:[container_port] [docker_image]". E.g. "docker run --name [give a name to container] --mount source=ectl-vol, target=/app -d --restart unless-stopped -p 8501:8501 [image id]". More info : ([How to Use Docker Run Command with Examples](#)) ([docker run](#)).
10. Start the container
 - a. OPTION 1: Run Start command "docker start [container name]"
 - b. OPTION 2: Go to Docker Desktop > Container, you should see the container created in the previous step. Click the Start button and click the "open with browser" button. The application will open on the browser automatically.
11. You can visit the application via <http://localhost:8501/> or [http://\[Host IPv4 Address\]:8501/](http://[Host IPv4 Address]:8501/)

Docker Terminologies:

1. **Docker Image** - is a read-only template that contains a set of instructions for creating a container that can run on the Docker platform.
2. **Docker Container** - A container is a runnable instance of an image. Is an isolated, standalone, executable software package which includes the application and its dependencies (e.g. python libraries and configuration). Well-suited for securely running software like databases or web applications that need access to sensitive resources without giving access to every user on the system
3. **Dockerfile** - A Dockerfile contains all the code to set up a docker container from downloading the docker image to set the environment. You can think of it as describing the complete operating system installation of the system that you want to run.

4. **Docker Host** - A docker host is a machine that runs the Docker engine and allows users to connect remotely and access the engine.
5. **Dockerhub** - GitHub for your Docker images; you can set up Dockerhub to automatically build an image anytime you update your Dockerfile in GitHub. *Note: We are not using Dockerhub.

Cloud Implementation:

1. Register Azure Account
2. Claim free credit
3. Create resource group
4. Search Ubuntu Server on Azure Marketplace
5. [Create Ubuntu Linux on Azure using Azure Portal - Virtual Machines in Microsoft Azure Cloud](#)
6. [Creating Virtual Machine In Azure - Azure Virtual Machine Tutorial - Step by Step](#)
7. Connect to VM via RDP or SSH.